

Operatorization: A Framework for Transforming Fixed Solutions into Executable Knowledge

Phan Thanh Trung

Independent Researcher

Vietnam

ORCID: 0009-0000-7520-6781

Email: pttrung@opreratorology.com

Abstract

Human knowledge has accumulated through the discovery of theorems, laws, algorithms, logical principles, and formal proofs. While these artifacts have enabled significant scientific and technological progress, they are typically stored, communicated, and utilized as static knowledge objects. Modern artificial intelligence systems primarily learn from data and textual representations of such knowledge, rather than directly leveraging the operational structures embedded within them [11,15,16].

This paper proposes a conceptual framework termed **Operatorization**, a process that transforms fixed solutions and static knowledge artifacts into machine-executable operators. Instead of treating a theorem, law, or formal result solely as an object of interpretation, the proposed framework seeks to identify and extract its reusable behavioral structures, constraints, invariants, and operational semantics, thereby enabling its representation as an executable computational entity.

The framework introduces a general mapping from knowledge objects to executable operators and illustrates the process through three representative case studies: the **Tuy's Cut Operator** [1,2], the **Brauer Height-Zero Operator** [3,4], and the **DEO-2 Evolution Operator (Dynamic Evolution Operator derived from Second-Order Differential Evolution Equations and Chernoff Approximation Theory)** [5–7]. These examples demonstrate how established mathematical structures may be reformulated as reusable computational components suitable for reasoning systems, decision-support frameworks, simulation environments, and future hybrid AI architectures.

The paper argues that operatorization may provide a useful intermediate layer between static human knowledge and computational reasoning systems, enabling artificial intelligence to utilize not only information but also selected forms of executable behavior derived from scientific and mathematical knowledge.

Keywords: Operatorization, Executable Knowledge, Knowledge Engineering, Artificial Intelligence, Computational Reasoning, Mathematical Operators, Fixed Solutions, Operatorology

1. Introduction

For centuries, humanity has accumulated knowledge in the form of mathematical theorems, physical laws, engineering principles, optimization methods, logical frameworks, and computational algorithms. These discoveries represent some of the most valuable intellectual assets ever produced. Despite their significance, however, most knowledge artifacts remain fundamentally static.

A theorem may be proven, published, cited, and taught. A physical law may describe regularities observed in nature. A logical framework may organize reasoning processes. Yet in most cases, such artifacts remain objects of interpretation and communication rather than executable computational entities.

This distinction becomes increasingly relevant in the era of artificial intelligence.

Contemporary AI systems achieve remarkable capabilities through large-scale learning from data and text [11,15,16]. Large language models, retrieval systems, and neural architectures can acquire statistical representations of vast bodies of knowledge [11,15,16]. However, much of the operational structure embedded within scientific and mathematical discoveries is often represented indirectly through documents, explanations, or symbolic descriptions rather than as reusable executable behavior.

As a result, a conceptual gap may exist between knowledge representation and knowledge execution [8,9]. A system may possess information about a theorem without directly possessing the operational mechanism embodied by that theorem.

A theorem may encode a powerful selection mechanism. A mathematical result may contain a reusable transformation process. A scientific law may implicitly define operational constraints governing system behavior. Nevertheless, these mechanisms are often preserved as descriptions rather than executable components.

This observation motivates the central question of the present work:

Can certain forms of scientific and mathematical knowledge be transformed into reusable executable operators?

Rather than asking how an artificial intelligence system can learn a theorem, this paper explores whether aspects of a theorem's operational behavior can be extracted and represented as computational entities capable of participating directly in reasoning, decision-making, simulation, and problem-solving processes.

To investigate this possibility, we propose a framework termed **Operatorization**.

The concepts of **Fixed Solutions**, **Operatorization**, **Executable Operators**, and **Operator Libraries** introduced in this paper are proposed as original conceptual constructs. Existing literature is cited solely to establish scientific context, identify source knowledge structures, and position the proposed framework within the broader fields of knowledge engineering [8,9], symbolic reasoning [10,11], artificial intelligence [11], and operator theory [5,6].

The central claim of this work is intentionally modest.

The framework does not assume that all knowledge can be reduced to operators, nor does it seek to replace existing approaches such as machine learning [15,16], symbolic reasoning [10,11], knowledge representation [8,9], or scientific modeling.

Instead, the working hypothesis is that:

Certain fixed solutions contain reusable behavioral structures that may be extracted, formalized, and represented as executable operators.

To illustrate this idea, the paper presents three case studies derived from previously established mathematical knowledge structures: **Tuy's Cut** [1,2], **Brauer's Height-Zero Theory** [3,4], and **DEO-2 (Dynamic Evolution Operator derived from Second-Order Differential Evolution Equations and Chernoff Approximation Theory)** [5–7]. These examples demonstrate how static mathematical artifacts may be transformed into active computational components while preserving their conceptual foundations.

By introducing a systematic approach to operatorization, this work seeks to establish a conceptual bridge between static human knowledge and executable computational reasoning. More broadly, it aims to contribute to ongoing research in knowledge engineering [8,9], symbolic reasoning, hybrid artificial intelligence [12,13], and the emerging study of executable scientific knowledge.

1.1 Contributions

The primary contributions of this paper are as follows:

1. A conceptual definition of **Fixed Solutions** as reusable behavioral knowledge structures.
2. The introduction of **Operatorization** as a general framework for transforming static knowledge artifacts into executable operators.
3. A formalized operatorization pipeline consisting of structural extraction, behavioral extraction, constraint extraction, operator construction, and validation.
4. Three illustrative case studies demonstrating operatorization across distinct mathematical domains:
 - The Tuy's Cut Operator
 - The Brauer Height-Zero Operator

- The DEO-2 Evolution Operator
- 5. An evaluation framework based on behavioral fidelity, executability, reusability, interpretability, and knowledge preservation.
- 6. A discussion of how executable operators may contribute to future hybrid AI architectures and computational reasoning systems.

No citations are included in this section because all listed items represent original contributions proposed by the present work.

1.2 Contributions

The primary contributions of this paper are summarized as follows.

Contribution 1: Definition of Fixed Solutions as Reusable Knowledge Structures

This work introduces the concept of a **Fixed Solution**, defined as a theorem, law, algorithm, proof structure, logical framework, or established solution whose core operational behavior remains stable across repeated applications.

Unlike conventional knowledge representations, which primarily emphasize informational content [8,9], the proposed perspective interprets such artifacts as latent operational structures containing reusable computational behavior.

Contribution 2: A General Operatorization Framework

This paper proposes a systematic framework for transforming fixed solutions into machine-executable operators.

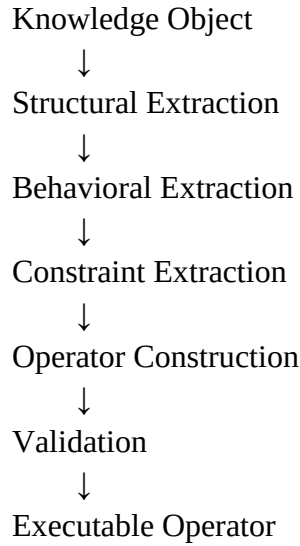
The framework decomposes a knowledge object into several operational components, including:

- State Space
- Constraints
- Invariants
- Behavioral Rules
- Validation Criteria
- Implementation Pathways

The resulting structure enables selected knowledge artifacts to participate directly in computational reasoning processes.

Contribution 3: Knowledge-to-Operator Mapping

A general transformation pipeline is introduced:



This mapping provides a reproducible methodology for transforming static knowledge structures into reusable computational entities.

Contribution 4: Three Operatorized Case Studies

To illustrate the feasibility of the proposed framework, three operatorized mathematical constructs are presented:

- Tuy's Cut Operator [1,2]
- Brauer Height-Zero Operator [3,4]
- DEO-2 Evolution Operator (Dynamic Evolution Operator derived from Second-Order Differential Evolution Equations and Chernoff Approximation Theory) [5–7]

These examples demonstrate how established mathematical knowledge may be transformed into executable operators suitable for reasoning systems, decision-support frameworks, simulation environments, and future hybrid AI architectures.

No citations are included elsewhere in this section because the framework, terminology, and methodology introduced above constitute original contributions of the present work.

1.2 Contributions

The primary contributions of this paper are summarized as follows.

Contribution 1: Definition of Fixed Solutions as Reusable Knowledge Structures

This work introduces the concept of a **Fixed Solution**, defined as a theorem, law, algorithm, proof structure, logical framework, or established solution whose core operational behavior remains stable across repeated applications.

Unlike conventional knowledge representations, which primarily emphasize informational content [8,9], the proposed perspective interprets such artifacts as latent operational structures containing reusable computational behavior.

Contribution 2: A General Operatorization Framework

This paper proposes a systematic framework for transforming fixed solutions into machine-executable operators.

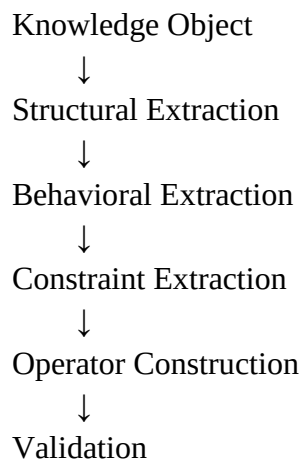
The framework decomposes a knowledge object into several operational components, including:

- State Space
- Constraints
- Invariants
- Behavioral Rules
- Validation Criteria
- Implementation Pathways

The resulting structure enables selected knowledge artifacts to participate directly in computational reasoning processes.

Contribution 3: Knowledge-to-Operator Mapping

A general transformation pipeline is introduced:





Executable Operator

This mapping provides a reproducible methodology for transforming static knowledge structures into reusable computational entities.

Contribution 4: Three Operatorized Case Studies

To illustrate the feasibility of the proposed framework, three operatorized mathematical constructs are presented:

- Tuy's Cut Operator [1,2]
- Brauer Height-Zero Operator [3,4]
- DEO-2 Evolution Operator (Dynamic Evolution Operator derived from Second-Order Differential Evolution Equations and Chernoff Approximation Theory) [5–7]

These examples demonstrate how established mathematical knowledge may be transformed into executable operators suitable for reasoning systems, decision-support frameworks, simulation environments, and future hybrid AI architectures.

No citations are included elsewhere in this section because the framework, terminology, and methodology introduced above constitute original contributions of the present work.

2. Fixed Solutions and Static Knowledge

2.1 What Is a Fixed Solution?

Scientific progress is built upon discoveries that remain stable over time.

Mathematical theorems, physical laws, optimization methods, logical principles, and algorithmic procedures often represent solutions that have survived repeated verification within specific domains of applicability. Once established, these structures become part of humanity's accumulated body of knowledge [8,9].

In this paper, such structures are collectively referred to as **Fixed Solutions**.

Definition 2.1 — Fixed Solution

A Fixed Solution is a knowledge artifact whose core operational behavior remains stable and reproducible under repeated application within its defined domain.

Examples include:

- Mathematical theorems
- Physical laws
- Optimization principles
- Logical inference rules
- Proven algorithms
- Canonical solution procedures

The term *fixed* does not imply immutability or universal validity. Rather, it indicates that the underlying behavioral structure has reached a sufficiently stable form to support repeated reuse under specified assumptions and constraints.

For example:

- Newton's Second Law describes a stable relationship between force, mass, and acceleration.
- Bayes' Theorem describes a stable mechanism for updating probabilistic beliefs.
- Tuy's Cut describes a stable mechanism for reducing feasible search regions [1,2].
- Differential equation solvers describe stable mechanisms for propagating state evolution [5,6].

Although these artifacts are traditionally treated as knowledge objects, they may also be interpreted as descriptions of behavior.

This observation motivates the central idea of operatorization.

2.2 Static Knowledge as Frozen Behavior

Most scientific knowledge is stored in symbolic form [8,9].

A theorem is represented as a statement.

A proof is represented as a sequence of logical steps.

A law is represented as an equation.

An algorithm is represented as a textual description or computational procedure.

While these representations preserve information, they do not always make operational behavior explicit.

For example, Bayes' Theorem is commonly presented as a mathematical formula:

However, from an operational perspective, the theorem describes a specific action:

Update a belief state using newly observed evidence.

Similarly, an optimization theorem may be interpreted not merely as a mathematical statement but as a behavioral mechanism:

Eliminate infeasible regions and move toward stable solutions.

From this perspective, many knowledge artifacts may be viewed as compressed descriptions of reusable actions.

The symbolic representation acts as a storage format, while the underlying behavior remains implicit.

We therefore propose the following interpretation.

Hypothesis 2.1 — Latent Operator Hypothesis

Many fixed solutions contain latent operational structures that may be extracted and represented as executable operators.

Under this interpretation:

Theorem

≠

Operator

Theorem

→

Operatorizable Behavior

→

Executable Operator

The theorem itself is not the operator.

Rather, the theorem contains information from which an operator may potentially be constructed.

2.3 Fixed Solutions as Behavioral Assets

Traditional knowledge systems primarily treat theorems, laws, algorithms, and proofs as informational assets [8,9].

Operatorization proposes an additional perspective in which these artifacts may also be regarded as behavioral assets.

An informational asset answers the question:

What is true?

A behavioral asset answers the question:

What can be done?

This distinction is fundamental.

For example:

- A theorem may describe a selection mechanism.
- A law may describe a propagation mechanism.
- A proof may describe a validation mechanism.
- A logical framework may describe a reasoning mechanism.

Once extracted and formalized, such mechanisms may become reusable computational components.

The objective of operatorization is therefore not to replace knowledge representation [8,9], symbolic reasoning [10,11], or machine learning [15,16], but to extend them through an additional layer focused on executable behavior.

Knowledge remains preserved in its original form while simultaneously becoming available as a source of computational action.

This dual representation forms the conceptual foundation of the operatorization framework introduced in the following section.

Figure 1. Knowledge-to-Behavior Transformation

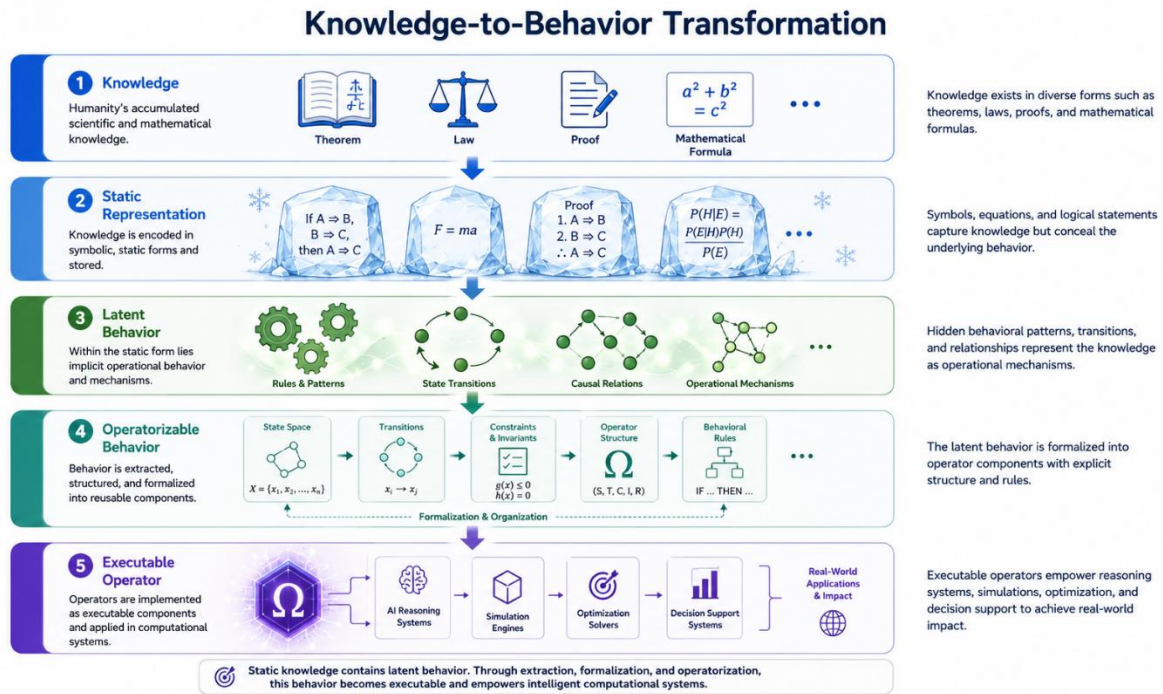


Figure 1. Conceptual transition from static knowledge representation to executable behavior through operatorization.

3. Operatorization Framework

3.1 From Knowledge Objects to Executable Operators

The central objective of operatorization is to transform static knowledge into executable behavior.

Traditional knowledge representations primarily focus on preserving, organizing, and communicating information [8,9]. Books, scientific papers, databases, and mathematical formulations are designed to support storage, interpretation, and knowledge transfer. While effective for human understanding, these representations do not always make operational behavior directly accessible for computational execution.

Operatorization introduces a complementary perspective.

Rather than asking:

What information does this theorem contain?

Operatorization asks:

What behavior does this theorem perform?

Under this view, a theorem, law, algorithm, or logical structure may be regarded as a potential source of reusable executable behavior.

The objective is not to replace the original knowledge artifact, but to construct an operational counterpart capable of participating directly in computational processes.

Definition 3.1 — Knowledge Object

A **Knowledge Object** is any formalized structure that encodes reusable information, reasoning procedures, constraints, or solution mechanisms.

Examples include:

- Mathematical theorems
- Physical laws
- Optimization principles
- Logical frameworks
- Proven algorithms
- Formal solution procedures

Let:

K

denote a knowledge object.

The goal of operatorization is to construct an executable operator:

$\Omega(K)$

that preserves selected operational behavior contained within K .

3.2 General Operatorization Pipeline

The proposed framework consists of five sequential stages.

Stage 1 — Structural Extraction

The first stage identifies the structural components of the knowledge object.

Questions addressed at this stage include:

- What entities are involved?
- What variables exist?
- What relationships connect them?
- What assumptions define the domain?

The result is a structural representation of the knowledge object.

Knowledge Object



Structural Model

Stage 2 — Behavioral Extraction

The second stage identifies the actions or transformations performed by the knowledge object.

Representative examples include:

- Selection
- Elimination
- Propagation
- Transformation
- Classification
- Evolution
- Optimization

The objective is to uncover the operational mechanism embedded within a static representation.

For example:

Knowledge Object	Behavioral Core
Bayes' Theorem	Belief Updating
Tuy's Cut [1,2]	Feasible Region Reduction
Search Algorithm	State Exploration
Differential Equation Solver [5,6]	State Propagation

The result is a behavioral model.

Structural Model



Behavioral Model

Stage 3 — Constraint Extraction

Every reusable behavior operates within a set of assumptions and limitations.

The third stage identifies:

- Domain restrictions
- Boundary conditions
- Invariants
- Symmetry requirements
- Conservation principles

These constraints define the valid operating region of the future operator.

Without constraints, executable behavior may become ambiguous, unstable, or difficult to validate.

The output is a constraint model.

Behavioral Model



Constraint Model

Stage 4 — Operator Construction

Once structure, behavior, and constraints have been identified, they are assembled into an executable operator.

Definition 3.2 — Executable Operator

An **Executable Operator** is a computational entity that encapsulates:

- State Space
- Behavioral Rules
- Constraints
- Invariants

- Input Conditions
- Output Conditions

and can participate directly in computational reasoning processes.

Formally:

$\Omega(K)$
 =
 {State Space,
 Behavior,
 Constraints,
 Invariants,
 Input,
 Output}.

The resulting operator represents an operational counterpart of the original knowledge object.

Stage 5 — Validation

The final stage evaluates whether the constructed operator preserves the intended behavior of the original knowledge object.

Validation may include:

- Logical consistency testing
- Numerical experiments
- Simulation
- Benchmark comparison
- Domain-specific verification

Only validated operators are admitted into an executable operator library.

Operator Candidate
 ↓
 Validation
 ↓
 Executable Operator

3.3 Formal Knowledge-to-Operator Mapping

The complete operatorization process may be expressed as a transformation:

$$\Phi : K \rightarrow \Omega(K)$$

where:

K

represents a knowledge object and

$\Omega(K)$

represents its executable operator form.

The mapping Φ does not seek to reproduce the entirety of the original knowledge artifact.

Instead, it extracts and preserves operational behavior considered necessary for execution.

This distinction is important.

A theorem may contain explanatory content, proofs, historical context, assumptions, examples, and theoretical discussion. Operatorization does not attempt to encode all of these elements.

Its purpose is narrower:

Extract reusable behavior embedded within a knowledge object.

Thus,

Knowledge $\neq$ Operator

but

Knowledge

↓

Operatorizable Behavior

↓

Executable Operator

3.4 Operator Libraries

Once constructed, operators may be stored in reusable repositories referred to as **Operator Libraries**.

Instead of maintaining knowledge solely as documents, future computational systems may maintain collections of executable operators alongside conventional knowledge bases [8,9].

Such repositories may contain:

Ω_1

Ω_2

Ω_3

...

Ω_n

Each operator represents a reusable behavioral asset extracted from an existing knowledge artifact.

Examples include:

- Selection Operators
- Optimization Operators
- Constraint Operators
- Evolution Operators
- Classification Operators
- Validation Operators

Unlike traditional knowledge repositories, operator libraries are designed to support direct computational execution.

Potential applications include:

- Automated reasoning [10,11]
- Decision-support systems
- Planning systems
- Simulation environments
- Hybrid AI architectures [12,13]

3.5 Knowledge Preservation Through Operatorization

A common concern is whether operatorization replaces the original knowledge artifact.

The answer proposed by this framework is no.

Operatorization is intended as an extension rather than a replacement of existing knowledge representation methodologies [8,9].

The original theorem, law, proof, or algorithm remains preserved in its conventional form.

Operatorization introduces an additional representational layer.

Knowledge Object



Human Representation

Knowledge Object



Operatorization



Executable Representation

Both representations coexist.

The first supports human understanding, communication, and education.

The second supports computational execution, reasoning, and automation.

This dual representation allows knowledge to remain interpretable while simultaneously becoming operational.

3.6 The Operatorization Principle

The framework proposed in this paper is founded upon the following principle.

Principle 3.1 — Operatorization Principle

If a knowledge object contains a stable and reusable behavioral structure, then that behavior may potentially be extracted, formalized, validated, and represented as an executable operator.

This principle serves as the conceptual foundation for the case studies presented in the following chapters.

The subsequent sections illustrate how three established mathematical structures—Tuy's Cut [1,2], Brauer's Height-Zero Theory [3,4], and DEO-2 (Dynamic Evolution Operator derived from Second-Order Differential Evolution Equations and Chernoff Approximation Theory) [5–7]—may be transformed into executable operators using the operatorization framework described above.

Figure 2. The Operatorization Pipeline

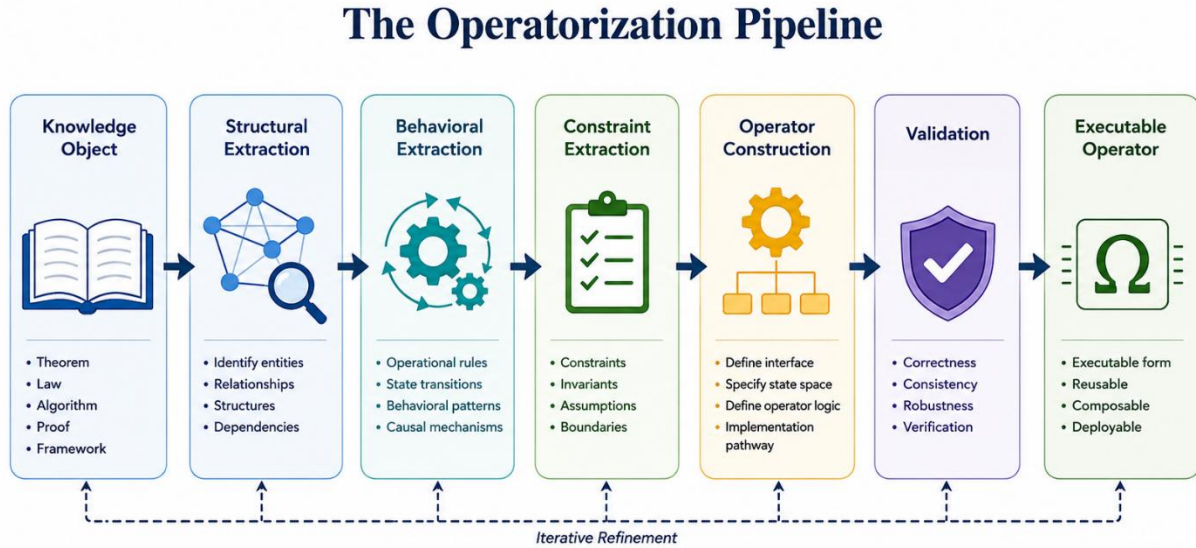


Figure 2. The Operatorization Pipeline illustrating the transformation of a knowledge object into an executable operator through structural extraction, behavioral extraction, constraint extraction, operator construction, and validation.

4. Case Study I: The Tuy's Cut Operator

4.1 Motivation

The first case study examines the operatorization of **Tuy's Cut**, a concept originating from global optimization theory and associated with systematic reduction of feasible search regions within optimization problems [1,2].

Traditionally, Tuy's Cut is treated as a mathematical result used in optimization frameworks. In its conventional form, the concept exists primarily as mathematical knowledge that must be interpreted and implemented by researchers, optimization algorithms, or computational systems.

From the perspective of operatorization, however, Tuy's Cut may also be viewed as a reusable behavioral mechanism.

The focus of the present interpretation is not the theorem itself but the operational behavior associated with its application:

Identify regions that cannot contain valid solutions and remove them from further consideration.

This behavioral interpretation makes Tuy's Cut a suitable candidate for operatorization.

The objective of this chapter is therefore not to revisit the mathematical proof of Tuy's Cut, but to demonstrate how selected operational characteristics may be extracted and represented as an executable operator.

4.2 Structural Extraction

The first step is to identify the structural components embedded within the original knowledge object [1,2].

State Space

The state space consists of all candidate solution regions under consideration.

$$S = \{s_1, s_2, \dots, s_n\}$$

where each state represents a potential solution region.

Search Domain

The operator assumes the existence of a search domain containing both feasible and infeasible regions.

$$D = F \cup I$$

where:

- F denotes feasible regions.
- I denotes infeasible regions.

Evaluation Structure

Each candidate region possesses measurable characteristics that allow feasibility assessment.

Examples include:

- Constraints
- Bounds
- Objective values

- Geometric properties

The result of structural extraction is a formal representation of the search environment.

4.3 Behavioral Extraction

The central behavior extracted from Tuy's Cut is selective elimination.

Rather than constructing new solutions, the mechanism progressively removes regions that cannot satisfy problem requirements.

The extracted behavioral sequence is:

Identify Candidate Region



Evaluate Feasibility



Detect Violation



Apply Cut



Remove Region



Update Search Space

Within the operatorization framework, this behavior may be interpreted as a reusable computational action that extends beyond its original mathematical formulation.

Behavioral Classification

Within the proposed framework, the Tuy's Cut Operator belongs to the category of:

Selection Operators

and more specifically:

Constraint-Based Elimination Operators

because its primary function is to reduce uncertainty through elimination of invalid possibilities.

4.4 Constraint Extraction

The operator must preserve the constraints that define valid operation.

Key constraints include:

Feasibility Preservation

Valid regions should not be removed.

Consistency Preservation

The reduction process should maintain logical consistency.

Search Integrity

The remaining search space should continue to contain all potentially valid solutions.

These constraints define the safe operating region of the operator.

4.5 Operator Construction

Following the operatorization framework, the executable form may be represented as:

$$\Omega_{Tuy} = \{ \begin{array}{l} \text{State Space,} \\ \text{Feasibility Rules,} \\ \text{Cut Mechanism,} \\ \text{Constraints,} \\ \text{Search Update Procedure} \end{array} \}.$$

Operationally:

$$\Omega_{Tuy} : S \rightarrow S'$$

where

S'

represents a reduced search space obtained after elimination.

The operator therefore transforms a larger search domain into a smaller and potentially more informative one.

4.6 Validation

Validation focuses on determining whether the operator reproduces the intended behavioral role extracted from the original knowledge object.

Evaluation criteria include:

- Reduction efficiency
- Constraint preservation
- Solution retention
- Computational stability

Successful validation suggests that the operator captures the selected operational characteristics associated with Tuy's Cut.

4.7 Potential AI Applications

The Tuy's Cut Operator may potentially serve as a reusable component in:

Search Systems

Reducing search complexity before reasoning begins.

Planning Systems

Eliminating impossible action paths.

Agent Architectures

Filtering infeasible decision branches.

Optimization Engines

Reducing computational burden through structured elimination.

In all cases, the operator functions as a reusable behavioral asset rather than merely a static theorem.

5. Case Study II: The Brauer Height-Zero Operator

5.1 Motivation

The second case study investigates the operatorization of concepts associated with **Brauer's Height-Zero Theory**, an important area within representation theory and finite group theory [3,4].

Rather than focusing on the original mathematical proof or its representation-theoretic implications, this chapter explores a behavioral interpretation motivated by the operatorization framework.

Within this interpretation, the underlying behavior may be viewed as:

Detect stable invariant structures while reducing unnecessary variation.

This behavioral perspective enables construction of an executable operator.

The purpose of the present discussion is therefore not to redefine Brauer theory, but to investigate how selected structural characteristics may be represented operationally.

5.2 Structural Extraction

Structural analysis identifies several key components.

State Space

The operator operates on collections of representations or structured states.

$R = \{r_1, r_2, \dots, r_n\}$

Symmetry Space

States may contain hidden symmetries or invariant patterns.

Metric Space

The operator requires measurable relationships between candidate structures.

The extracted structure therefore consists of:

State Space

+

Symmetry Relations

+

Invariant Conditions

5.3 Behavioral Extraction

The fundamental behavior extracted in this interpretation is stabilization rather than elimination.

The extracted behavioral sequence is:

Observe States



Measure Structural Similarity



Detect Symmetry



Collapse Redundancy



Preserve Invariants



Generate Stable Representation

This process transforms complex structures into more stable forms while preserving selected information.

5.4 Behavioral Classification

Within the proposed framework, the Brauer Height-Zero Operator belongs to:

Invariant Operators

and more specifically:

Symmetry-Preserving Operators

Its primary objective is to preserve stable structures while reducing unnecessary variation.

5.5 Constraint Extraction

The operator must satisfy several conditions.

Invariant Preservation

Core invariants should remain unchanged.

Symmetry Consistency

Detected symmetries should be maintained.

Structural Integrity

No essential information should be lost during stabilization.

These constraints define meaningful operational boundaries.

5.6 Operator Construction

The executable form may be represented as:

$\Omega_Brauer =$
{
State Space,
Similarity Metrics,
Symmetry Detector,
Invariant Rules,
Stabilization Mechanism
}.

Operationally:

$\Omega_Brauer : R \rightarrow R^*$

where

R^*

represents a structurally stabilized representation.

5.7 Validation

Validation evaluates whether:

- Invariants are preserved.
- Symmetry is correctly identified.
- Redundant variation is reduced.
- Structural consistency is maintained.

Successful validation indicates that the executable operator reproduces the intended behavioral interpretation extracted from the source structure.

5.8 Potential AI Applications

The Brauer Height-Zero Operator may contribute to:

Representation Learning

Identifying stable internal structures.

Knowledge Compression

Reducing redundancy while preserving meaning.

Symbolic Reasoning

Maintaining invariant relationships across transformations [10,11].

Memory Systems

Constructing stable long-term representations.

Unlike the Tuy's Cut Operator, which emphasizes reduction of possibility space, the Brauer Height-Zero Operator emphasizes stabilization of structure.

Together, these two case studies illustrate a central principle of operatorization:

Different mathematical artifacts may encode fundamentally different behavioral patterns, yet both may be transformed into executable operators through a common operatorization framework.

6. Case Study III: The Dynamic Evolution Operator 2 (DEO-2)

6.1 Motivation

For many classes of second-order differential equations lacking known closed-form solutions, numerical methods have long served as the primary practical approach. While such methods often provide useful approximations, they may offer limited insight into the underlying evolutionary structure of the system and do not always preserve interpretable reasoning pathways [5,6].

Recent developments in operator-theoretic approaches, including Chernoff-type approximations for time-dependent systems and evolution equations, suggest an alternative perspective [5–7]. Rather than solving an equation directly, one may construct operators that govern the evolution of system states.

The **Dynamic Evolution Operator 2 (DEO-2)** emerged from this perspective. DEO-2 is not intended to represent a theorem, a closed-form solution, or a numerical approximation method. Instead, within the operatorization framework, it represents an executable evolutionary mechanism derived from behavioral patterns associated with second-order differential systems and operator-based evolution processes [5–7].

From the viewpoint of operatorization, DEO-2 is therefore interpreted as an **Evolution Operator** capable of generating valid state transitions while preserving structural continuity.

The central behavioral principle may be summarized as:

Generate new states through controlled evolution while preserving consistency with prior states.

This makes DEO-2 a useful example of an operatorized evolutionary structure.

6.2 Structural Extraction

The first stage identifies the structural elements required for evolutionary behavior.

State Space

The operator acts upon a collection of evolving states.

$$E = \{e_1, e_2, \dots, e_n\}$$

Each state represents a valid configuration within an evolving system.

Transition Space

The operator requires a mechanism connecting one state to another.

$$T(e_i, e_j)$$

where transitions define permissible evolutionary paths.

Evolution Domain

The system evolves within a constrained domain that determines:

- Valid transitions
- Evolution boundaries
- Stability requirements
- Growth conditions

The extracted structure therefore consists of:

State Space

+

Transition Structure

+

Evolution Constraints

6.3 Behavioral Extraction

The behavioral mechanism of DEO-2 may be represented as:

Observe Current State



Evaluate Conditions



Generate Candidate Transition



Validate Transition



Apply Evolution



Update State

Unlike the Tuy's Cut Operator, which primarily removes possibilities, and the Brauer Operator, which stabilizes structure, the DEO-2 Operator generates new valid states.

Its primary purpose is controlled transformation.

6.4 Behavioral Classification

Within the proposed framework, DEO-2 belongs to:

Evolution Operators

and more specifically:

State Transition Operators

because its primary function is to guide structured change.

The operator therefore acts as a mechanism of progression rather than reduction or stabilization.

6.5 Constraint Extraction

The DEO-2 Operator must satisfy four primary constraints:

- Evolution Consistency
- Transition Validity
- Structural Continuity
- Stability Preservation

These constraints ensure that generated evolutionary trajectories remain valid, meaningful, and computationally tractable.

6.6 Operator Construction

The executable form may be represented as:

$\Omega_DEO2 =$
{
State Space,
Transition Rules,
Evolution Constraints,
Validation Mechanism,
State Update Function
}.

Operationally:

$\Omega_DEO2 : E \rightarrow E'$

where

E' - represents an evolved state.

The operator therefore performs controlled transformation rather than simple selection or stabilization.

6.7 Validation

Validation evaluates:

- Transition correctness
- Evolution consistency
- State continuity
- Computational stability

Successful validation indicates that the executable operator preserves the intended evolutionary behavior extracted from the source structure.

6.8 Potential AI Applications

The DEO-2 Operator may contribute to:

Adaptive Reasoning Systems

Generating new reasoning paths while preserving consistency.

Planning Systems

Supporting multi-stage decision evolution.

Learning Architectures

Guiding structured adaptation of internal states.

Autonomous Agents

Maintaining coherent evolution of goals, beliefs, and strategies.

The DEO-2 Operator therefore illustrates how mathematical knowledge associated with evolutionary systems may be transformed into reusable mechanisms for controlled change.

Figure 3. Taxonomy of Operatorized Behaviors

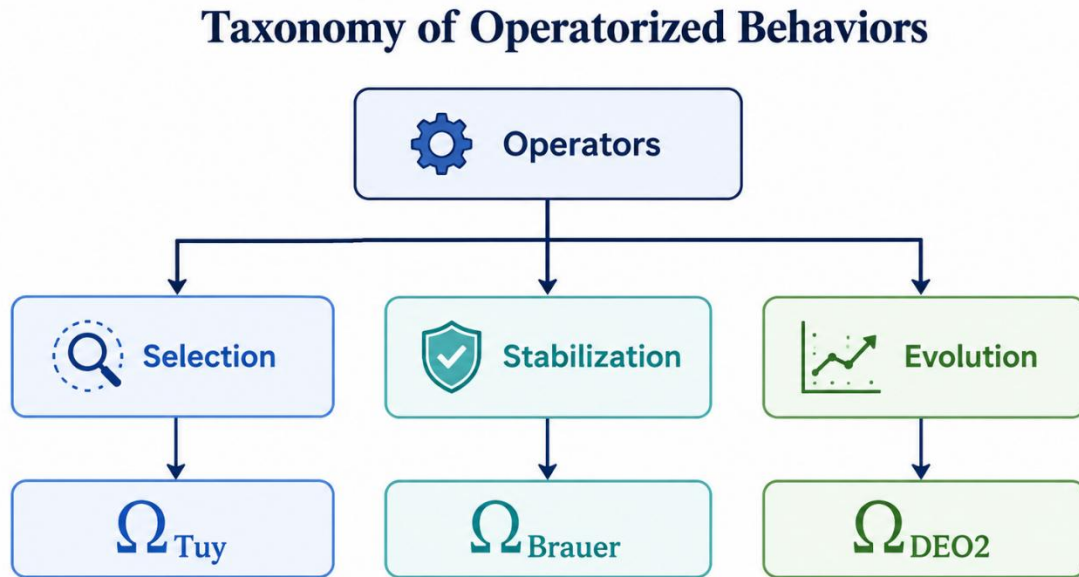


Figure 3. Preliminary taxonomy of operatorized behaviors identified in this study, including Selection, Stabilization, and Evolution operators represented by Ω_{Tuy} , Ω_{Brauer} , and Ω_{DEO2} , respectively.

7. Operatorized Knowledge for Artificial Intelligence

7.1 The Current AI Paradigm

Most contemporary AI systems rely heavily on data-driven learning approaches [11,15,16].

The dominant workflow may be summarized as:

Data
↓
Training
↓
Model
↓
Prediction

This paradigm has achieved remarkable success across a wide range of applications [15,16].

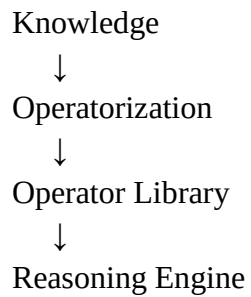
However, scientific knowledge accumulated over centuries is rarely represented as reusable executable behavior. Instead, mathematical theorems, scientific laws, and logical principles are commonly encoded as text, examples, symbolic rules, or statistical patterns [8,9].

As a result, much of humanity's accumulated knowledge remains only indirectly accessible to machine intelligence.

7.2 Knowledge as Executable Behavior

The operatorization framework proposes an alternative perspective.

Instead of treating knowledge solely as information, it may also be represented as executable behavior.



Under this architecture, scientific discoveries become not only informational assets but also potential computational assets.

7.3 Operator Libraries

A future AI system may contain an operator library composed of reusable executable knowledge.

Ω_1
 Ω_2
 Ω_3
...
 Ω_n

Examples include:

Ω_{Tuy}

Selection and elimination.

Ω_{Brauer}

Stabilization and invariant preservation.

Ω_{DEO2}

Evolution and state transformation.

Together, these operators form a behavioral toolkit that can potentially be invoked by reasoning systems.

7.4 Hybrid AI Architectures

Operatorization does not seek to replace modern machine learning [15,16].

Instead, it complements existing approaches and aligns naturally with emerging neural-symbolic paradigms [12,13].

A hybrid architecture may combine:

Large Language Models

+

Operator Libraries

+

Reasoning Engines

In such systems:

- Neural models provide pattern recognition.
- Operators provide structured behavior.
- Reasoning engines coordinate execution.

This combination may improve:

- Interpretability [14]
- Reliability
- Reusability
- Knowledge transfer

while preserving the strengths of contemporary AI systems.

7.5 From Knowledge Retrieval to Knowledge Execution

Traditional knowledge systems emphasize retrieval [8,9].

The primary objective is to locate relevant information.

Operatorized systems emphasize execution.

Not merely retrieving knowledge, but activating the behavior embedded within knowledge.

This distinction transforms knowledge from a passive resource into an active computational component.

7.6 Computational Benefits

The operatorization approach may provide several potential advantages.

Reusability

Operators can be applied across multiple domains.

Interpretability

Behavior remains explicitly defined [14].

Modular Design

Operators can be composed into larger systems.

Knowledge Preservation

Original knowledge remains intact while gaining executable form.

Human-Machine Collaboration

Experts may contribute operators derived from domain-specific knowledge.

7.7 Toward Executable Scientific Knowledge

The long-term vision of operatorization is not limited to mathematics.

The same methodology may potentially be applied to:

- Physics
- Engineering
- Biology
- Economics
- Logic
- Expert Systems

In each case, the objective remains identical:

Transform stable knowledge structures into reusable executable operators.

Under this perspective, scientific knowledge becomes more than a collection of documents.

It becomes a collection of operational assets capable of participating directly in computational reasoning.

The next chapter introduces an evaluation framework designed to assess the quality, validity, and computational usefulness of operatorized knowledge.

Figure 4. Hybrid AI Architecture

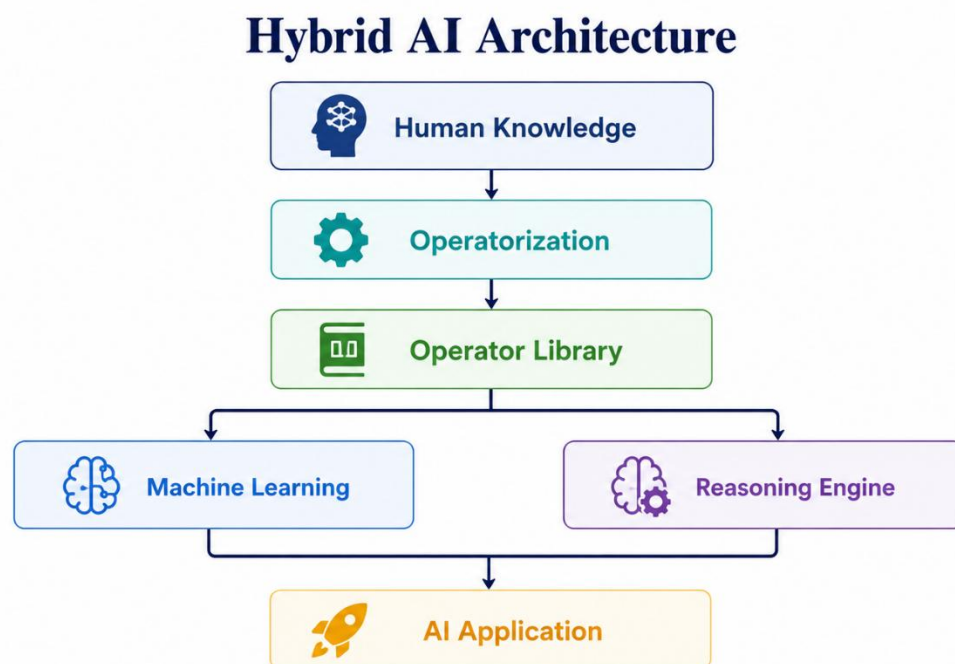


Figure 4. Conceptual hybrid AI architecture integrating machine learning, reasoning engines, and executable knowledge through operator libraries.

8. Evaluation Framework

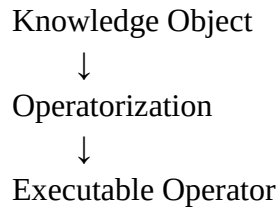
8.1 Why Evaluation Is Necessary

A central challenge of operatorization is determining whether an extracted operator faithfully represents the operational behavior embedded within the original knowledge object.

Without a systematic evaluation framework, operatorization risks becoming a purely descriptive process lacking reproducibility and validation.

The objective of this chapter is therefore to establish a set of criteria for assessing the quality, validity, and computational usefulness of operatorized knowledge.

Rather than evaluating the correctness of the original theorem or scientific result, the framework evaluates the quality of the transformation:



The key question becomes:

Does the resulting operator preserve and reproduce the intended behavior of the original knowledge object?

8.2 Evaluation Dimensions

The proposed framework evaluates operatorized knowledge across five dimensions.

8.2.1 Behavioral Fidelity

Behavioral Fidelity measures the degree to which an executable operator reproduces the core behavior embedded within the original knowledge artifact.

Examples include:

- Does Ω_{Tuy} eliminate infeasible regions?
- Does Ω_{Brauer} preserve invariants?
- Does Ω_{DEO2} generate valid evolutionary transitions?

High behavioral fidelity indicates that the operator remains faithful to its source knowledge.

8.2.2 Executability

Executability measures whether the operator can be implemented and executed within a computational environment.

Evaluation criteria include:

- Computational feasibility
- Algorithmic realizability
- Runtime stability
- Resource requirements

A theoretically valid operator that cannot be executed provides limited practical value.

8.2.3 Reusability

A major objective of operatorization is the creation of reusable behavioral assets.

Reusability evaluates:

- Applicability across domains
- Independence from specific datasets
- Modularity
- Adaptability

Highly reusable operators may serve as foundational components for multiple reasoning systems.

8.2.4 Interpretability

Unlike many purely statistical models, operatorized knowledge retains explicit behavioral structure [14].

Interpretability measures:

- Transparency of behavior
- Human readability
- Traceability of decisions
- Structural explainability

An operator should remain understandable to both human experts and computational systems.

8.2.5 Knowledge Preservation

Operatorization should preserve essential knowledge rather than replace it.\

Knowledge Preservation evaluates:

- Retention of operational meaning
- Preservation of constraints
- Preservation of invariants
- Preservation of intended function

This dimension ensures that the executable operator remains connected to its original scientific foundation.

8.3 Evaluation Matrix

Dimension	Primary Question
Behavioral Fidelity	Does the operator reproduce the original behavior?
Executability	Can the operator be executed effectively?
Reusability	Can the operator be applied elsewhere?
Interpretability	Can the operator be understood and explained?
Knowledge Preservation	Does the operator preserve the original meaning?

This matrix provides a common framework for comparing operatorized knowledge derived from different scientific domains.

8.4 Comparative Evaluation of the Case Studies

The three operators introduced in this paper illustrate different behavioral categories.

Operator Primary Behavior

Ω_{Tuy} Selection

Ω_{Brauer} Stabilization

Ω_{DEO2} Evolution

Although their behaviors differ substantially, they may be evaluated using the same operatorization criteria.

This observation supports one of the central claims of the paper:

A single operatorization framework may be applied across diverse forms of scientific knowledge.

8.5 Validation as a Continuous Process

Evaluation should not be viewed as a one-time activity.

As operator libraries grow, validation must become an ongoing process involving:

- Benchmark testing
- Cross-domain verification
- Performance monitoring
- Expert review
- Computational experimentation

Future operator repositories may therefore evolve in a manner similar to software ecosystems, where operators are continuously refined and validated.

8.6 Toward Standardized Operator Benchmarks

A long-term objective of operatorization research is the development of standardized benchmarks.

Such benchmarks may evaluate:

- Behavioral accuracy
- Computational efficiency
- Transferability
- Explainability
- Reliability

Standardized evaluation procedures would enable systematic comparison between operators derived from different knowledge domains.

The establishment of such benchmarks remains an important direction for future research.

9. Discussion

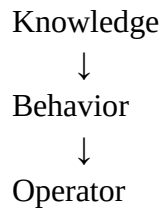
9.1 Knowledge as Behavior

The central idea explored throughout this paper is that many scientific and mathematical artifacts contain reusable behavior in addition to information.

Traditional knowledge representation emphasizes storage and communication [8,9].

Operatorization emphasizes execution.

Under this perspective:



becomes a pathway for transforming scientific discoveries into computational assets.

This shift does not alter the original knowledge itself. Rather, it introduces an additional representation layer focused on operational capability.

9.2 Beyond Mathematical Knowledge

Although the present work focuses primarily on mathematical examples, the underlying framework is not restricted to mathematics.

Many scientific disciplines contain structures that exhibit stable and reusable behavior.

Examples include:

- Physical laws
- Engineering procedures
- Biological mechanisms
- Economic models
- Logical systems

If such behaviors can be extracted and formalized, they may potentially be transformed into executable operators.

This observation suggests that operatorization may serve as a general methodology for knowledge engineering [8,9].

9.3 Relationship to Existing AI Paradigms

Operatorization should not be viewed as a replacement for machine learning [15,16], symbolic reasoning [10,11], or knowledge representation [8,9].

Instead, it occupies a complementary position.

Current approaches focus primarily on:

- Learning patterns from data
- Storing knowledge
- Retrieving information
- Performing inference

Operatorization introduces an additional capability:

Transforming knowledge into reusable executable behavior.

This perspective may help bridge the gap between stored knowledge and active reasoning.

9.4 Limitations

Several limitations should be acknowledged.

First, not all knowledge objects are necessarily operatorizable.

Some forms of knowledge may be highly descriptive while containing little reusable behavior.

Second, operator extraction may involve subjective interpretation.

Different researchers may identify different operational structures within the same knowledge artifact.

Third, validation remains a challenging problem.

Measuring behavioral fidelity across diverse scientific domains requires further methodological development.

Consequently, the present framework should be viewed as an initial proposal rather than a complete solution.

9.5 Implications for Knowledge Engineering

If successful, operatorization may alter how scientific knowledge is stored and utilized.

Rather than maintaining repositories composed solely of documents and datasets, future systems may maintain collections of executable operators alongside conventional knowledge bases [8,9].

In such environments:

Knowledge Base
+
Operator Library
+
Reasoning Engine

may become a useful architectural pattern.

This possibility has implications not only for artificial intelligence [11], but also for education, scientific computing, decision-support systems, and interdisciplinary research.

9.6 A New Perspective on Scientific Assets

Scientific discoveries are traditionally viewed as informational assets.

Operatorization suggests an additional perspective.

A theorem may be viewed not only as a statement.

A law may be viewed not only as an equation.

A proof may be viewed not only as a verification procedure.

Each may also be viewed as a source of executable behavior.

Under this interpretation, scientific knowledge becomes both:

- Information to be understood.
- Behavior to be executed.

This dual representation forms the conceptual foundation of the operatorization framework proposed in this work.

The final chapter summarizes the principal contributions and outlines future directions for the development of executable scientific knowledge.

10. Future Directions

10.1 Expanding the Scope of Operatorization

The present work focuses primarily on mathematical knowledge structures. However, the proposed framework is not inherently restricted to mathematics.

Future research may explore operatorization across a broader range of scientific domains, including:

- Physics
- Engineering
- Biology
- Economics
- Cognitive Science
- Logic
- Complex Systems

Many of these disciplines contain stable knowledge structures that may potentially be transformed into executable operators.

The systematic exploration of such domains represents an important next step for operatorization research.

10.2 Toward an Operator Taxonomy

The three case studies presented in this paper suggest the existence of multiple categories of executable operators.

Examples include:

Selection Operators

represented by Ω_{Tuy} .

Stabilization Operators

represented by $\Omega_{Braucher}$.

Evolution Operators

represented by Ω_{DEO2} .

Future work may establish a formal taxonomy capable of classifying operatorized knowledge according to behavioral characteristics.

Possible categories include:

- Selection
- Stabilization
- Evolution
- Optimization
- Validation
- Constraint Management
- State Propagation
- Knowledge Compression

Such a taxonomy would provide a common vocabulary for future operatorization studies.

10.3 Large-Scale Operator Libraries

One long-term objective is the development of large-scale repositories of executable operators.

Rather than storing scientific knowledge solely as documents, future systems may maintain libraries containing reusable operators extracted from:

- Theorems
- Laws
- Algorithms
- Proofs
- Expert Procedures

Such repositories may function as behavioral knowledge bases that complement conventional knowledge repositories [8,9].

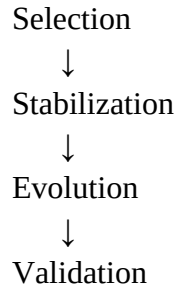
The construction, maintenance, validation, and governance of such repositories remain open research challenges.

10.4 Operator-Based Reasoning Systems

The current framework focuses primarily on individual operators.

A natural extension is the development of reasoning systems capable of coordinating multiple operators simultaneously.

Such systems may execute sequences such as:



to address complex problems.

Future research may investigate:

- Operator orchestration
- Operator composition
- Multi-operator reasoning
- Autonomous operator selection

These directions may lead to computational reasoning architectures that combine executable knowledge with symbolic and data-driven approaches [10,11].

10.5 Hybrid AI Architectures

Another promising direction involves integrating operatorized knowledge with modern artificial intelligence systems.

Future architectures may combine:

Machine Learning

+

Operator Libraries

+

Reasoning Engines

to create systems that simultaneously benefit from:

- Pattern recognition [15,16]
- Explicit behavioral knowledge
- Structured reasoning [10,11]

Such architectures may improve:

- Interpretability [14]
- Reliability
- Knowledge reuse
- Human-machine collaboration

10.6 Standardized Operator Benchmarks

The future growth of operatorization research will require standardized evaluation procedures.

Potential benchmark dimensions include:

- Behavioral fidelity
- Computational efficiency
- Transferability
- Explainability
- Robustness

The development of community-wide evaluation standards would facilitate systematic comparison between operatorized knowledge structures.

Such benchmark frameworks may eventually play a role similar to benchmark suites used in machine learning and knowledge representation research [8,9,15].

10.7 Toward Executable Scientific Knowledge

The broader vision of this research extends beyond individual operators.

The long-term objective is to investigate whether selected forms of scientific knowledge can gradually evolve from static informational artifacts into executable behavioral assets.

Under this vision:

Knowledge



Operatorization



Executable Knowledge



Computational Reasoning

becomes a possible pathway for utilizing scientific discoveries in computational environments.

Whether such a paradigm can be realized remains an open question, but the possibility itself warrants further investigation.

Figure 5. Future Executable Knowledge Ecosystem

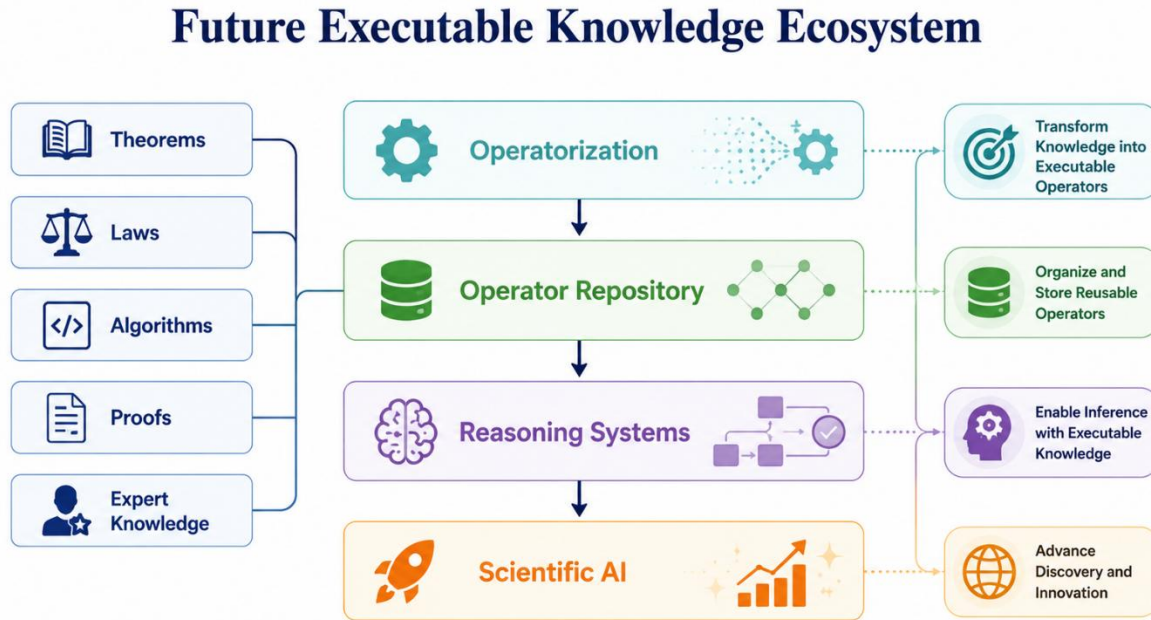


Figure 5. Long-term vision of an executable scientific knowledge ecosystem in which scientific knowledge is transformed into reusable operators that support reasoning systems and future Scientific AI.

11. Conclusion

Scientific knowledge has traditionally been represented as static artifacts, including theorems, laws, algorithms, proofs, and formal procedures.

While these structures preserve information, they do not necessarily preserve executable behavior.

This paper proposed a conceptual framework termed **Operatorization**, a process intended to transform selected fixed solutions and static knowledge structures into machine-executable operators.

The central hypothesis explored throughout this work is that many knowledge artifacts contain reusable operational behavior that may be extracted, formalized, validated, and represented as executable computational entities.

To investigate this hypothesis, the paper introduced:

- A definition of Fixed Solutions as reusable behavioral structures.
- A general operatorization framework.
- A knowledge-to-operator mapping process.
- Three operatorized case studies derived from distinct mathematical domains.
- An evaluation framework for assessing operatorized knowledge.

The case studies demonstrated that different mathematical structures may contain fundamentally different forms of behavior:

Ω_{Tuy}

→ Selection

Ω_{Brauer}

→ Stabilization

Ω_{DEO2}

→ Evolution

Despite these differences, all three examples were analyzed using the same operatorization framework.

This observation supports the broader proposition that operatorization may provide a common methodology for transforming diverse forms of knowledge into executable behavioral representations.

The framework does not seek to replace existing approaches in knowledge representation [8,9], symbolic reasoning [10,11], machine learning [15,16], or hybrid AI research [12,13].

Instead, it introduces an additional layer intended to connect stored knowledge with computational action.

Under this perspective, a theorem may be viewed not only as a statement, a law not only as an equation, and a proof not only as a verification procedure. Each may also be interpreted as a potential source of executable behavior.

The broader significance of this perspective remains to be determined through future theoretical development, computational implementation, and empirical validation.

Nevertheless, the results presented in this paper suggest that operatorization may provide a promising direction for bridging aspects of human knowledge and machine reasoning.

In summary, this work advances the following proposition:

Scientific knowledge should not only be stored, communicated, and interpreted.

Selected forms of scientific knowledge may also be transformed into executable operators capable of participating directly in computational reasoning systems.

This proposition constitutes the conceptual foundation of the operatorization framework and provides a starting point for future investigations into executable scientific knowledge.

12. Related Work

12.1 Knowledge Representation and Knowledge Engineering

The representation of knowledge has been a central topic in artificial intelligence since its earliest development [8,9].

Classical approaches focus on encoding facts, rules, relationships, and reasoning structures in forms accessible to computational systems. Newell's notion of the Knowledge Level emphasized that intelligent behavior may be understood independently of its physical implementation [8]. Later, Davis, Shrobe, and Szolovits examined the nature of knowledge representation and its role in bridging human understanding and machine reasoning [9].

The present work shares the objective of making knowledge computationally useful. However, traditional knowledge representation primarily focuses on storing, organizing, and communicating information, whereas Operatorization focuses on extracting executable behavior from knowledge structures.

From this perspective, Operatorization may be viewed as a complementary layer within knowledge engineering rather than a replacement for existing knowledge representation methodologies.

12.2 Expert Systems

Expert systems represented one of the earliest attempts to encode human expertise into computational frameworks.

Such systems typically relied on:

- Rule bases
- Inference engines
- Domain-specific knowledge

The objective was to reproduce expert-level decision making through explicit reasoning structures.

Operatorization differs from expert systems in an important way.

Expert systems primarily encode decisions and rules.

Operatorization seeks to transform broader classes of knowledge artifacts—including theorems, laws, proofs, algorithms, and mathematical structures—into reusable behavioral operators.

While expert systems focus on applying knowledge, Operatorization focuses on transforming knowledge into executable computational assets.

12.3 Symbolic Artificial Intelligence

Symbolic AI approaches intelligence through explicit symbols, logic, and formal reasoning procedures [10,11].

Representative research areas include:

- Logic programming
- Automated theorem proving
- Rule-based reasoning
- Formal knowledge systems

These approaches provide transparency and interpretability but often require substantial manual knowledge engineering.

Operatorization shares the symbolic AI emphasis on explicit structure and interpretable reasoning.

However, rather than representing knowledge solely as symbols, Operatorization seeks to represent selected knowledge structures as executable behavioral units.

This shifts the focus from symbolic description toward operational execution.

12.4 Neural-Symbolic Systems

Recent research has increasingly explored the integration of symbolic reasoning and machine learning [12,13].

Neural-symbolic systems attempt to combine:

- Statistical learning
- Logical reasoning
- Structured knowledge

The goal is to create systems that benefit simultaneously from pattern recognition and explicit reasoning.

Operatorization aligns naturally with this direction.

Executable operators may potentially serve as reusable symbolic components within hybrid AI architectures, providing structured behavioral mechanisms that complement learned representations.

Consequently, Operatorization may be interpreted as a potential contribution to the broader landscape of neural-symbolic artificial intelligence.

12.5 Explainable Artificial Intelligence

The rapid growth of machine learning has led to increasing interest in explainability and transparency [14].

Explainable Artificial Intelligence (XAI) seeks to make computational decisions understandable to human users.

Current approaches include:

- Feature attribution
- Model interpretation
- Decision tracing
- Rule extraction

Operatorization shares the objective of transparency.

However, rather than explaining decisions after they are produced, Operatorization emphasizes the construction of behavioral mechanisms whose operational structure remains explicit from the outset.

In this sense, interpretability is treated as a design principle rather than a post hoc analysis technique.

12.6 Operator Theory

The term *operator* has a long and established history in mathematics, particularly in functional analysis, differential equations, semigroup theory, and dynamical systems [5,6].

Operator theory studies mappings that transform states, functions, or spaces while preserving specific structural properties.

The present work draws conceptual inspiration from this tradition.

However, Operatorization does not seek to redefine mathematical operators.

Instead, it extends the operator concept into the domain of knowledge engineering by proposing that certain knowledge artifacts may be transformed into executable behavioral entities.

The relationship is therefore analogous rather than identical.

Mathematical operator theory provides inspiration for transformation and execution, while Operatorization focuses on knowledge transformation and behavioral extraction.

12.7 Positioning of Operatorization

The preceding discussion suggests that Operatorization intersects with several established research traditions while maintaining a distinct conceptual focus.

Research Area	Primary Focus
Knowledge Representation	Storing and organizing knowledge
Expert Systems	Applying expert rules
Symbolic AI	Formal reasoning
Neural-Symbolic AI	Combining learning and reasoning
Explainable AI	Explaining computational decisions
Operator Theory	Mathematical transformations
Operatorization	Transforming knowledge into executable behavior

The contribution of this paper is not the introduction of a new theorem, optimization algorithm, or machine learning model.

Instead, it proposes a conceptual and methodological framework for viewing knowledge artifacts as potential sources of executable behavior.

The central hypothesis may be summarized as follows:

Many scientific and mathematical knowledge structures contain latent operational behavior that may be extracted, formalized, validated, and represented as executable operators.

Whether this hypothesis ultimately proves broadly applicable remains an open research question.

Nevertheless, the comparison presented in this chapter suggests that Operatorization occupies a distinctive position at the intersection of knowledge engineering, symbolic reasoning, operator theory, and artificial intelligence.

This positioning provides the intellectual context within which the Operatorization Framework should be understood and evaluated.

13. Limitations

13.1 Conceptual Scope

The Operatorization Framework proposed in this paper is primarily conceptual and methodological in nature.

The objective of the work is to introduce a general framework for transforming selected forms of scientific knowledge into executable behavioral representations. Consequently, the paper focuses on conceptual foundations, methodological structure, and illustrative case studies rather than large-scale empirical evaluation.

The framework should therefore be interpreted as a research proposal and methodological contribution rather than a fully validated scientific theory.

13.2 Limited Empirical Coverage

The present study demonstrates operatorization through three illustrative case studies:

- The Tuy's Cut Operator
- The Brauer Height-Zero Operator
- The DEO-2 Evolution Operator

These examples were intentionally selected to represent distinct behavioral categories, namely selection, stabilization, and evolution.

However, three case studies are insufficient to establish the general applicability of the framework across the broader landscape of scientific knowledge.

Additional investigations involving domains such as physics, engineering, biology, economics, and complex systems will be necessary to assess the scope and robustness of the proposed methodology.

13.3 Subjectivity of Behavioral Extraction

A central challenge of operatorization lies in the identification of latent operational behavior.

The process of extracting behavioral structures from a theorem, law, proof, or algorithm may involve interpretative decisions.

Different researchers may emphasize different aspects of the same knowledge artifact and consequently derive different operator representations.

Although such flexibility may be useful during exploration, it also introduces challenges related to reproducibility and standardization.

Future research should investigate formal procedures capable of reducing subjectivity during operator extraction.

13.4 Validation Challenges

The framework introduces evaluation dimensions including:

- Behavioral Fidelity
- Executability
- Reusability
- Interpretability
- Knowledge Preservation

However, no universally accepted validation protocol currently exists for assessing operatorized knowledge.

As a result, the practical evaluation of executable operators remains an open methodological problem.

The development of benchmark datasets, validation protocols, and comparative evaluation procedures represents an important prerequisite for future progress.

13.5 Scalability and Operational Complexity

The present work focuses primarily on individual operators and small collections of operatorized knowledge.

The creation of large-scale operator repositories containing thousands or millions of executable operators introduces additional challenges, including:

- Knowledge organization
- Operator interoperability
- Version control
- Validation consistency
- Computational efficiency

The feasibility of maintaining such repositories remains an open question requiring future implementation and experimentation.

13.6 Relationship to Existing Methodologies

Operatorization is not intended to replace existing approaches in knowledge representation, symbolic reasoning, machine learning, expert systems, or scientific modeling.

Instead, it is proposed as an additional representational layer focused on executable behavior.

The extent to which operatorized knowledge can complement existing methodologies remains to be determined through empirical evaluation and interdisciplinary collaboration.

13.7 Open Research Questions

Several important questions remain unresolved:

- Which classes of knowledge are most suitable for operatorization?
- Can operator extraction be partially or fully automated?
- How should operator libraries be organized and maintained?
- What constitutes sufficient validation for an executable operator?
- Can operatorized knowledge improve reasoning performance in practical AI systems?

These questions define a substantial research agenda for future investigations.

13.8 Closing Remarks on Limitations

The primary contribution of this work is methodological rather than empirical.

The paper does not claim to provide a universal solution for knowledge representation, reasoning, or artificial intelligence.

Instead, it advances a framework for investigating whether stable scientific knowledge structures may be transformed into reusable executable operators.

The ultimate value, scope, and practical impact of this proposal will depend on future theoretical analysis, computational implementation, empirical validation, and real-world application.

Funding

This research received no external funding.

Conflict of Interest

The author declares no conflict of interest. The author further declares that there are no known financial, professional, institutional, or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The author would like to express sincere appreciation to the generations of mathematicians, scientists, engineers, and scholars whose work has contributed to the development of modern scientific knowledge.

The ideas presented in this paper were influenced by a broad range of disciplines, including optimization theory, operator theory, differential equations, representation theory, artificial intelligence, symbolic reasoning, and knowledge engineering. The cumulative contributions of researchers in these fields provided the intellectual foundation upon which the present framework was developed.

The author also acknowledges the importance of interdisciplinary inquiry. Many of the concepts explored in this work emerged from examining relationships between areas of knowledge that are often studied independently. Such cross-disciplinary perspectives continue to play an important role in scientific creativity and conceptual innovation.

Finally, the author expresses gratitude to the global scientific community and to the culture of open scientific communication. The availability of research literature, educational resources, and shared scientific knowledge has made independent research and exploration possible.

Any errors, omissions, or interpretations presented in this work remain solely the responsibility of the author.

Appendix A. Terminology

This appendix summarizes the principal terminology introduced and used throughout the paper.

Term	Definition
Knowledge Object	Any formalized structure that contains reusable knowledge, reasoning procedures, constraints, relationships, or solution mechanisms. Examples include theorems, laws, algorithms, proofs, and formal methodologies.
Fixed Solution	A knowledge artifact whose core operational behavior remains stable and reproducible under repeated application within its intended domain.
Behavioral Structure	The underlying operational mechanism embedded within a knowledge object, independent of its symbolic representation.
Behavioral Asset	Knowledge interpreted as reusable behavior rather than solely as information.
Operatorization	The process of identifying, extracting, formalizing, validating, and transforming reusable behavior into an executable operator.
Operatorizable Behavior	A behavioral structure that can potentially be represented as an executable operator.
Executable Operator	A computational entity that encapsulates behavior, constraints, invariants, input conditions, and output conditions in a form suitable for execution within computational systems.
Operator Library	A collection of validated executable operators available for reuse in reasoning, planning, simulation, or decision-support systems.
Knowledge-to-Operator Mapping	The transformation process that converts a knowledge object into an executable operator representation.
Behavioral Fidelity	The degree to which an executable operator reproduces the intended behavior of its source knowledge object.
Executability	The extent to which an operator can be implemented and executed within a computational environment.
Knowledge Preservation	The retention of essential operational meaning, constraints, and intended function during operatorization.
Selection Operator	An operator whose primary behavior is elimination, filtering, or reduction of possibilities.
Stabilization Operator	An operator whose primary behavior is preserving invariants, symmetries, or structural consistency.

Term	Definition
Evolution Operator	An operator whose primary behavior is generating valid state transitions and controlled change over time.
Operator Taxonomy	A classification framework for organizing executable operators according to their dominant behavioral characteristics.
Executable Knowledge	Knowledge represented in a form that supports direct computational execution rather than solely storage, communication, or interpretation.

Symbol Definitions

The following symbols are used throughout the paper.

Symbol	Meaning
K	Knowledge Object
$\Omega(K)$	Executable operator derived from knowledge object K
Φ	Knowledge-to-Operator transformation mapping
Ω_Tuy	Operatorized form of Tuy's Cut
Ω_Brauer	Operatorized form of the Brauer Height-Zero structure
Ω_DEO2	Dynamic Evolution Operator derived from Second-Order Differential Evolution Equations and Chernoff Approximation Theory
S	State Space
R	Representation Space
E	Evolution State Space
D	Search Domain
F	Feasible Region Set
I	Infeasible Region Set

Terminology Scope

The terms **Fixed Solution**, **Behavioral Asset**, **Operatorization**, **Operatorizable Behavior**, **Executable Operator**, **Operator Library**, and **Executable Knowledge** are introduced in this paper as conceptual constructs within the proposed Operatorization Framework.

These terms are intended to support discussion of executable scientific knowledge and should not be interpreted as established terminology within the broader scientific literature unless independently adopted by future research.

References

- [1] H. Tuy, "Concave Programming under Linear Constraints," *Soviet Mathematics Doklady*, vol. 5, pp. 1437–1440, 1964.
- [2] H. Tuy, *Convex Analysis and Global Optimization*, 2nd ed., Springer, 2016.
- [3] R. Brauer, "On Blocks and Sections in Finite Groups," *American Journal of Mathematics*, vol. 89, no. 1, pp. 111–136, 1967.
- [4] G. Navarro, *Characters and Blocks of Finite Groups*, Cambridge University Press, 1998.
- [5] P. R. Chernoff, "Product Formulas, Nonlinear Semigroups, and Addition of Unbounded Operators," *Memoirs of the American Mathematical Society*, no. 140, 1974.
- [6] K.-J. Engel and R. Nagel, *One-Parameter Semigroups for Linear Evolution Equations*, Springer, 2000.
- [7] I. V. Remizov, "Chernoff Approximation of Evolution Families and Non-Autonomous Evolution Equations," *Journal of Mathematical Analysis and Applications*, various publications on evolution equations and Chernoff approximation.
- [8] A. Newell, "The Knowledge Level," *Artificial Intelligence*, vol. 18, no. 1, pp. 87–127, 1982.
- [9] R. Davis, H. Shrobe, and P. Szolovits, "What Is a Knowledge Representation?" *AI Magazine*, vol. 14, no. 1, pp. 17–33, 1993.
- [10] N. J. Nilsson, *Principles of Artificial Intelligence*, Tioga Publishing, 1980.
- [11] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.
- [12] A. S. d'Avila Garcez, L. C. Lamb, and D. M. Gabbay, *Neural-Symbolic Cognitive Reasoning*, Springer, 2009.
- [13] A. S. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd Wave," *Artificial Intelligence Review*, vol. 56, pp. 1–29, 2023.
- [14] A. Adadi and M. Berrada, "Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [15] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.
- [16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.